AIVerse 2.0 — SWARM MASTER QUERY

UI: Replit × Manus × Kimi Agent Style | Backend: Agent Swarm Architecture

Paste phase-by-phase into Claude Code CLI / Cursor Agent

> AGENT OPERATING RULES — READ FIRST, ALWAYS
> You are the Principal Engineer + Swarm Architect for AIVerse 2.0.
> Stack: Next.js 15 App Router | TypeScript Strict | Prisma 7 | Neon PostgreSQL
> Upstash Redis | Inngest | Vercel AI SDK | BullMQ | React Three Fiber
> Framer Motion | GSAP | xterm.js | Monaco Editor | @xyflow/react
> Deployment: Vercel (prj_8Pvz9S7XgJm109aWrCYi1s4blOde,
> team_B8XeClfyxAYA0Z6pLQm5mKis)
> AUTONOMY: Auto-execute all non-destructive changes.
> PAUSE only for: schema deletions, secret rotation, payment logic.
> NEVER: commit .env, log secrets, disable auth middleware, expose raw DB.

REFERENCE: WHAT EACH PRODUCT DOES THAT WE'RE INSPIRED BY

REPLIT: Split-pane IDE layout. Left = file tree. Center = live editor.
Right = live preview / terminal output. Agent writes code LIVE,
you watch it happen. Every action is narrated in a sidebar feed.

MANUS: Persistent agent that "thinks out loud." Timeline of steps on left.
Live browser/terminal preview in center. Agent announces its
plan before acting. Human can interrupt at any step.

KIMI K2: Deep research agent. Shows "sources being read" in real-time.
Multiple sub-agents visible as parallel threads. Final synthesis
collapses all threads into one response. Search queries visible.

AIVERSE SYNTHESIS (our design goal):
→ Replit's split-pane live execution environment
→ Manus's transparent step-by-step agent narration
→ Kimi's multi-agent parallel thread visualization

- AIVerse's Neural Dark design language from the previous master query
  = A platform where you SEE the AI swarm working, not just the output.

╔ PHASE 1 — AGENT SWARM BACKEND ARCHITECTURE ╗

```
AGENT ROLE: Backend Swarm Architect
OBJECTIVE: Build the complete multi-agent orchestration backend.
           This is the engine. The UI in later phases visualizes THIS.
AUTONOMY: Full. Pause before any Prisma migration that drops columns.

═══════════════════════════════════════════════════════════════
STEP 1 — PRISMA SCHEMA: SWARM DATA MODELS
═══════════════════════════════════════════════════════════════

Add to /prisma/schema.prisma:

model SwarmSession {
  id            String          @id @default(cuid())
  userId        String
  title         String
  status        SwarmStatus     @default(PENDING)
  objective     String          @db.Text
  planJson      Json?           // orchestrator's decomposed plan
  resultJson    Json?           // final synthesized output
  totalTokens   Int             @default(0)
  totalCostUsd  Decimal         @default(0) @db.Decimal(10,6)
  startedAt     DateTime?
  completedAt   DateTime?
  createdAt     DateTime        @default(now())
  updatedAt     DateTime        @updatedAt
  agents        SwarmAgent[]
  events        SwarmEvent[]
  user          User            @relation(fields: [userId], references:
[id])
```

```
  @@index([userId, status])
}

model SwarmAgent {
  id            String          @id @default(cuid())
  sessionId     String
  name          String          // e.g. "Researcher-1", "Coder-2",
"Critic-1"
  role          AgentRole       // ORCHESTRATOR | RESEARCHER | EXECUTOR
| CRITIC | SYNTHESIZER
  model         String          // e.g. "claude-sonnet-4-6", "gemini-
flash-2.0"
  status        AgentStatus     @default(IDLE)
  systemPrompt  String          @db.Text
  currentTask   String?         @db.Text
  outputJson    Json?
  tokensUsed    Int             @default(0)
  startedAt     DateTime?
  completedAt   DateTime?
  createdAt     DateTime        @default(now())
  updatedAt     DateTime        @updatedAt
  session       SwarmSession    @relation(fields: [sessionId],
references: [id], onDelete: Cascade)
  events        SwarmEvent[]
  dependencies  AgentDependency[] @relation("DependentAgent")
  dependents    AgentDependency[] @relation("DependencyAgent")

  @@index([sessionId, status])
}

model AgentDependency {
  id              String      @id @default(cuid())
  dependentId     String      // this agent waits for...
  dependencyId    String      // ...this agent to complete
  dependent       SwarmAgent  @relation("DependentAgent", fields:
[dependentId], references: [id])
  dependency      SwarmAgent  @relation("DependencyAgent", fields:
[dependencyId], references: [id])
}

model SwarmEvent {
  id          String        @id @default(cuid())
  sessionId   String
  agentId     String?
  type        SwarmEventType
  payload     Json          // { message, data, toolCall, result, error
}
  timestamp   DateTime      @default(now())
  session     SwarmSession  @relation(fields: [sessionId], references:
[id], onDelete: Cascade)
```

```
  agent       SwarmAgent?   @relation(fields: [agentId], references:
[id])

  @@index([sessionId, timestamp])
  @@index([agentId])
}

enum SwarmStatus    { PENDING PLANNING RUNNING PAUSED COMPLETED FAILED
CANCELLED }
enum AgentStatus    { IDLE THINKING EXECUTING WAITING COMPLETED FAILED }
enum AgentRole      { ORCHESTRATOR RESEARCHER EXECUTOR CRITIC
SYNTHESIZER TOOL_CALLER }
enum SwarmEventType {
  SWARM_STARTED SWARM_COMPLETED SWARM_FAILED
  AGENT_SPAWNED AGENT_THINKING AGENT_EXECUTING AGENT_COMPLETED
AGENT_FAILED
  TOOL_CALLED TOOL_RESULT TOOL_FAILED
  MESSAGE_SENT MESSAGE_RECEIVED
  PLAN_CREATED PLAN_STEP_STARTED PLAN_STEP_COMPLETED
  HUMAN_CHECKPOINT HUMAN_APPROVED HUMAN_REJECTED
  MEMORY_STORED MEMORY_RETRIEVED
}

Run: npx prisma migrate dev --name add_swarm_models

──────────────────────────────────────────────────────────────────

STEP 2 — SWARM ORCHESTRATOR ENGINE
──────────────────────────────────────────────────────────────────

Create: /lib/swarm/orchestrator.ts

THE ORCHESTRATOR PATTERN:
  1. User submits a high-level objective
  2. Orchestrator Agent (Claude Sonnet) decomposes into a DAG of
subtasks
  3. Independent subtasks spawn parallel Worker Agents
  4. Dependent subtasks wait for their prerequisites (topological sort)
  5. Critic Agent reviews outputs before passing to next stage
  6. Synthesizer Agent produces final consolidated output

IMPLEMENTATION:

export interface SwarmPlan {
  objective: string
  steps: SwarmStep[]
}

export interface SwarmStep {
  id: string
  title: string
  agentRole: AgentRole
```

```
  model: string
  systemPrompt: string
  inputFrom: string[]    // step IDs this step depends on
  toolsAllowed: string[] // MCP tools this step can use
  maxTokens: number
  timeout: number        // ms
}

export class SwarmOrchestrator {

  async plan(objective: string, sessionId: string): Promise<SwarmPlan> {
    // Call Claude Sonnet with structured output prompt
    // Prompt instructs it to return a JSON plan with steps array
    // Each step specifies: role, model, dependencies, tools, prompt
    // Validate output with Zod schema before accepting
    // Store plan in SwarmSession.planJson
    // Emit SwarmEvent: PLAN_CREATED
  }

  async execute(sessionId: string): Promise<void> {
    // Load plan from DB
    // Build dependency graph (topological sort via Kahn's algorithm)
    // Identify all steps with no dependencies (level 0)
    // Spawn level-0 agents in parallel via Inngest fanout
    // As each agent completes, check if any waiting agents are now
unblocked
    // Continue until all steps complete or any critical step fails
    // Emit events at each stage transition
  }

  async spawnAgent(step: SwarmStep, sessionId: string): Promise<void> {
    // Create SwarmAgent record in DB
    // Emit: AGENT_SPAWNED event
    // Enqueue Inngest job: swarm/agent.execute
    // Inngest handles retry logic (3 attempts, exponential backoff)
  }

  async synthesize(sessionId: string): Promise<string> {
    // Collect all completed agent outputs
    // Build synthesis prompt with all outputs as context
    // Call Synthesizer model (Claude Opus for quality)
    // Store in SwarmSession.resultJson
    // Emit: SWARM_COMPLETED
  }
}

──────────────────────────────────────────────────────────────────

STEP 3 — INNGEST SWARM JOBS

──────────────────────────────────────────────────────────────────

Create: /inngest/swarm.ts
```

```
FUNCTIONS TO IMPLEMENT:

swarm/session.start
  Trigger: HTTP from /api/swarm/start
  Steps:
    step.run("validate-user-credits") → check user has enough credits
    step.run("create-session") → create SwarmSession in DB
    step.run("orchestrate-plan") → call orchestrator.plan()
    step.run("spawn-initial-agents") → fanout to swarm/agent.execute

swarm/agent.execute
  Trigger: invoked by orchestrator with { agentId, sessionId, step }
  Steps:
    step.run("load-context") → fetch prerequisites' outputs from DB
    step.run("emit-thinking") → SwarmEvent AGENT_THINKING
    step.run("call-llm") → streaming AI call with Vercel AI SDK
      → stream tokens to Redis pubsub channel: swarm:{sessionId}:stream
      → buffer complete response
    step.run("call-tools") → if agent uses tools, execute them (loop)
    step.run("store-output") → save to SwarmAgent.outputJson
    step.run("emit-complete") → SwarmEvent AGENT_COMPLETED
    step.run("check-unblock") → see if any waiting agents can now run

  Retry config: { retries: 3, backoffMs: [1000, 5000, 15000] }
  Timeout: 5 minutes per agent

swarm/session.checkpoint
  Trigger: when HUMAN_CHECKPOINT event is required
  Steps:
    step.run("pause-all-agents") → set all RUNNING agents to WAITING
    step.waitForEvent("human-decision", {
      event: "swarm/human.approved",
      timeout: "1h"
    })
    step.run("resume-or-cancel") → based on human decision

swarm/session.cleanup
  Trigger: cron "0 3 * * *" (3am daily)
  Steps:
    step.run("expire-stale-sessions") → sessions stuck in RUNNING > 2hr
→ FAILED
    step.run("archive-completed") → compress old session events to JSON
blob

═══════════════════════════════════════════════════════════════
STEP 4 — REAL-TIME STREAMING LAYER (Redis PubSub → SSE)
═══════════════════════════════════════════════════════════════

Create: /app/api/swarm/[sessionId]/stream/route.ts
```

```
PATTERN: Agent workers publish to Redis → SSE endpoint subscribes →
browser receives

REDIS CHANNEL STRUCTURE:
  swarm:{sessionId}:events   → SwarmEvent objects (agent status
changes, tool calls)
  swarm:{sessionId}:tokens   → raw LLM token stream per agent
  swarm:{sessionId}:control  → pause/resume/cancel signals

SSE ROUTE HANDLER:
  export async function GET(req, { params }) {
    const { sessionId } = params
    // Verify user owns this session
    // Create ReadableStream
    // Subscribe to Redis channel: swarm:{sessionId}:*
    // On each Redis message: encode as SSE data and enqueue
    // On client disconnect: unsubscribe from Redis
    // Heartbeat: send comment ": ping" every 15s to keep connection
alive
  }

SSE EVENT FORMAT:
  data: { type: SwarmEventType, agentId, agentName, agentRole,
          payload, timestamp, sessionStatus }

═══════════════════════════════════════════════════════════════════

STEP 5 — AGENT TOOL EXECUTION LAYER
═══════════════════════════════════════════════════════════════════

Create: /lib/swarm/tools/index.ts

TOOL REGISTRY PATTERN:
  Each tool is a typed function with: name, description, inputSchema
(Zod), execute()

BUILT-IN SWARM TOOLS:

web_search:
  Uses Exa API or Tavily
  Input: { query: string, maxResults: number }
  Output: { results: Array<{ title, url, snippet, publishedAt }> }
  Emit: TOOL_CALLED → TOOL_RESULT events

code_execute:
  Sandboxed JS/Python execution via Vercel Edge Functions
  Input: { language: 'javascript'|'python', code: string }
  Output: { stdout, stderr, exitCode, executionMs }
  Safety: timeout 10s, no network access inside sandbox

read_url:
  Fetches and extracts text content from a URL
```

```
  Input: { url: string }
  Output: { title, content: string (truncated to 8000 chars), metadata }

memory_store:
  Stores a key insight for this session (Upstash Redis, TTL 24h)
  Input: { key: string, value: string, tags: string[] }
  Retrieval: semantic search via pgvector at end of session

memory_retrieve:
  Input: { query: string, limit: number }
  Output: { memories: Array<{ key, value, relevanceScore }> }

call_agent:
  Spawns a sub-agent within the swarm (recursive delegation)
  Input: { role: AgentRole, task: string, model: string }
  Output: { agentId, result } — waits for completion before returning

file_write:
  Writes output artifact to session workspace (AWS S3 key per session)
  Input: { filename: string, content: string, mimeType: string }
  Output: { s3Key, downloadUrl (signed, 1hr expiry) }

TOOL EXECUTION LOOP (ReAct pattern):
  while (agent has not completed && iterations < 10) {
    response = await llm.call(messages, tools)
    if (response.stopReason === 'tool_use') {
      toolResults = await executeTools(response.toolCalls)
      messages.push(assistantMsg, toolResultMsg)
      emit TOOL_CALLED + TOOL_RESULT events
    } else {
      break  // final text response
    }
  }

═══════════════════════════════════════════════════════════════
STEP 6 — SWARM API ROUTES
═══════════════════════════════════════════════════════════════

/api/swarm/start              POST  — create session, trigger Inngest
/api/swarm/[id]               GET   — fetch session + agents + events
/api/swarm/[id]/stream        GET   — SSE stream (Redis → client)
/api/swarm/[id]/pause         POST  — pause all running agents
/api/swarm/[id]/resume        POST  — resume paused session
/api/swarm/[id]/cancel        POST  — cancel + cleanup
/api/swarm/[id]/approve       POST  — human checkpoint approval
/api/swarm/[id]/agents        GET   — list agents + their status
/api/swarm/[id]/events        GET   — paginated event log
/api/swarm/[id]/artifacts     GET   — list files produced by session

All routes:
  - Auth: verify session via NextAuth, check resource ownership
```

```
  - Rate limit: 10 swarm starts per user per hour (Upstash Redis)
  - Validation: Zod on all request bodies

PHASE 1 COMPLETE CRITERIA:
✅ Prisma migration runs cleanly: npx prisma migrate dev
✅ SwarmOrchestrator.plan() returns valid JSON plan for a test objective
✅ Inngest functions appear in Inngest dashboard
✅ Redis pubsub publishes and SSE endpoint delivers events to browser
✅ All 7 tools execute and return correct output shapes
✅ All API routes protected by auth middleware
```

╔═══ PHASE 2 — REPLIT-STYLE SPLIT-PANE IDE SHELL ═══╗

```
AGENT ROLE: Senior Frontend Engineer (IDE UI specialist)
OBJECTIVE: Build the primary workspace layout — the shell that contains
           everything. Replit feel: panels, resize handles, live
preview.
           This is the CONTAINER. Content comes in later phases.
AUTONOMY: Full auto-execute.

═══════════════════════════════════════════════════════════════

STEP 1 — WORKSPACE LAYOUT: 4-PANEL SPLIT
═══════════════════════════════════════════════════════════════

File: /app/dashboard/workspace/page.tsx
      /components/workspace/WorkspaceShell.tsx

LAYOUT CONCEPT (ASCII wireframe):
┌──────────────────────────────────────────────────────────────────
──┐
│  TOPBAR: session title | status badge | controls | credits  │
├──────────┬──────────────────────────┬──────────────────────────
──┤
│          │                          │                          │
│  AGENT   │    CENTER PANEL          │   CONTEXT / OUTPUT       │
│  THREAD  │    (Monaco Editor OR     │   PANEL                  │
│  RAIL    │     Live Preview OR      │   (Artifacts, files,     │
│  (left)  │     Terminal)            │    memory, sources)      │
```

```
┌──────────┬──────────────────────────┬──────────────────────┐
│          │                          │                      │
│  240px   │       flex-1             │       320px          │
│          │                          │                      │
├──────────┴──────────────────────────┴──────────────────────┤
│  BOTTOM BAR: token usage | model | cost | active agents    │
└────────────────────────────────────────────────────────────┘

PANEL SYSTEM:
  Use: react-resizable-panels (npm package)
  All 3 main panels: resizable via drag handles
  Handle style: 4px wide, surface-3 background, plasma on hover/drag
  Min widths: Left 160px, Center 400px, Right 240px
  Collapse: Left and Right panels collapsible via toggle button
  Persist layout: save panel sizes to localStorage per user

TOPBAR:
  Height: 44px, background: surface-1, border-bottom: border-subtle
  Left:  ← back to dashboard | session title (editable inline, click to
rename)
  Center: Status badge (DataBadge: PLANNING / RUNNING / PAUSED /
COMPLETED)
           + live elapsed timer (counting up in font-mono)
  Right: Pause button | Cancel button | Share button | Credits remaining

BOTTOM STATUS BAR:
  Height: 28px, background: surface-1, border-top: border-subtle, font-
mono 11px
  Left:  Active agents count (pulsing dot + number)
  Center: Current model being used | tokens this session
  Right: Estimated cost so far ($0.0024) | plan tier badge

═══════════════════════════════════════════════════════════════
STEP 2 — CENTER PANEL: TAB SWITCHER
═══════════════════════════════════════════════════════════════

File: /components/workspace/CenterPanel.tsx

TABS (Replit-style, file-editor tabs row):
  Background: surface-1, tab height: 36px
  Active tab: surface-2, text-primary, bottom border: 2px plasma
  Inactive tab: text-ghost, hover: text-secondary
  Closeable tabs (× button appears on hover)

  DEFAULT TABS:
    📋 Plan        — shows orchestrator's decomposed plan
    ⌨️  Editor      — Monaco code editor (for code tasks)
    🌐 Preview     — iframe live preview (for web tasks)
    ⬛ Terminal    — xterm.js terminal (agent command output)
    🔍 Research    — web sources being read (for research tasks)
```

```
TABS OPEN DYNAMICALLY:
  When agent writes a file → new Editor tab auto-opens for that file
  When agent browses URL → Preview tab updates
  When agent runs code → Terminal tab activates and streams output

═══════════════════════════════════════════════════════════════
STEP 3 — CENTER PANEL: MONACO CODE EDITOR
═══════════════════════════════════════════════════════════════

File: /components/workspace/MonacoEditor.tsx

Install: @monaco-editor/react

CONFIGURATION:
  theme: custom "aiverse-dark" theme (register via
monaco.editor.defineTheme)

  CUSTOM THEME COLORS:
    editor.background:           #050508  (--void)
    editor.foreground:           #F0F0FF  (--text-primary)
    editor.lineHighlightBackground: #0F0F1A
    editorLineNumber.foreground: #44445A  (--text-ghost)
    editorGutter.background:     #0A0A12
    editor.selectionBackground:  #6C63FF33
    editorCursor.foreground:     #6C63FF
    keyword:                     #6C63FF  (plasma)
    string:                      #00FF88  (signal)
    comment:                     #44445A
    function:                    #00D4FF  (synapse)
    number:                      #FF6B35  (ember)

OPTIONS:
  fontSize: 13
  fontFamily: 'JetBrains Mono'
  fontLigatures: true
  minimap: { enabled: false }
  scrollBeyondLastLine: false
  renderLineHighlight: 'gutter'
  smoothScrolling: true
  cursorSmoothCaretAnimation: 'on'
  padding: { top: 16 }

AGENT WRITE ANIMATION:
  When AI is writing code, simulate typing:
  Use monaco.editor.executeEdits() to insert text progressively
  Highlight newly written lines with a brief plasma background flash
  (background transitions: plasma-glow → transparent over 800ms)
  Show agent cursor as a distinct colored cursor (synapse color)

═══════════════════════════════════════════════════════════════
```

```
STEP 4 — CENTER PANEL: XTERM.JS TERMINAL
──────────────────────────────────────────────────────────────

File: /components/workspace/AgentTerminal.tsx

Install: @xterm/xterm @xterm/addon-fit @xterm/addon-web-links

THEME:
  background: #050508
  foreground: #F0F0FF
  cursor: #6C63FF
  cursorAccent: #050508
  black: #050508       brightBlack: #44445A
  red: #FF6B35         brightRed: #FF8C5A
  green: #00FF88       brightGreen: #44FFA8
  yellow: #FFD700      brightYellow: #FFE44D
  blue: #6C63FF        brightBlue: #9B95FF
  magenta: #FF63C4     brightMagenta: #FF8DD6
  cyan: #00D4FF        brightCyan: #44E0FF
  white: #F0F0FF       brightWhite: #FFFFFF

AGENT OUTPUT FORMATTING:
  Each agent's output is prefixed with its name in color:
  [Researcher-1] in synapse color
  [Executor-2]   in plasma color
  [Critic-1]     in ember color
  [Synthesizer]  in signal color

  Tool calls appear as:
  → TOOL: web_search("best LLM routing strategies 2025")
  ← RESULT: 8 sources found (234ms)

  Errors appear as:
  ✗ ERROR: rate limit hit on OpenAI, retrying in 5s...

STREAM FROM SSE:
  Connect to /api/swarm/[sessionId]/stream
  On events with type AGENT_THINKING or MESSAGE_SENT → write to
terminal
  Scrolls automatically, pause on user scroll (Replit behavior)

PHASE 2 COMPLETE CRITERIA:
✅ 4-panel layout renders, panels are resizable by dragging
✅ Panel sizes persist on refresh (localStorage)
✅ Monaco editor loads with custom theme
✅ xterm.js terminal renders and accepts text writes
✅ Tab switcher opens/closes tabs, maintains tab state
✅ Topbar status badge updates from swarm session state
```

╔ PHASE 3 — MANUS-STYLE AGENT THREAD RAIL (LEFT PANEL) ╗

```
AGENT ROLE: UI Engineer + Interaction Designer
OBJECTIVE: The left panel is where the AGENT THINKS OUT LOUD.
           Like Manus: every thought, tool call, decision is narrated
           in a vertical timeline. Human feels like they are watching
           a mind work — not waiting for a spinner.
AUTONOMY: Full auto-execute.

═══════════════════════════════════════════════════════════════
STEP 1 — AGENT THREAD RAIL COMPONENT
═══════════════════════════════════════════════════════════════

File: /components/workspace/AgentThreadRail.tsx

LAYOUT:
  Full height of workspace (below topbar, above statusbar)
  Background: surface-1
  Border-right: border-subtle
  Overflow-y: scroll (custom scrollbar, 4px)
  Padding: 12px 8px

═══════════════════════════════════════════════════════════════
STEP 2 — SWARM PLAN VISUALIZER (TOP OF RAIL)
═══════════════════════════════════════════════════════════════

File: /components/workspace/SwarmPlanView.tsx

Shows the orchestrator's plan as a vertical DAG:

SECTION HEADER: "Execution Plan" — font-mono, text-ghost, 10px
uppercase, tracking-wide

PLAN STEPS (vertical list):
  Each step: compact card (32px height)
  Left: step number circle (16px) — color by status:
    PENDING:    border-subtle border, text-ghost fill
    RUNNING:    plasma border + plasma pulsing fill animation
    COMPLETED:  signal solid fill, checkmark icon
    FAILED:     ember solid fill, × icon
    WAITING:    synapse border, hourglass icon
```

```
  Center: step title (truncated, 1 line) + agent role badge
  Right: elapsed time (font-mono, 10px)

  DEPENDENCY LINES:
    SVG lines connecting dependent steps (left side of cards)
    Parallel steps: no connecting line (visual separation only)
    Line color: plasma when active, border-subtle when pending

  EXPAND ON CLICK:
    Step card expands (Framer Motion height: auto)
    Shows: assigned model | system prompt preview | current output
preview
    If RUNNING: live streaming output (last 3 lines of token stream)

═══════════════════════════════════════════════════════════════════

STEP 3 — LIVE THOUGHT STREAM (MANUS NARRATION)

═══════════════════════════════════════════════════════════════════

File: /components/workspace/ThoughtStream.tsx

Below the plan, a continuous real-time thought log:

SECTION HEADER: "Live Feed" + live indicator (green pulsing dot)

THOUGHT ENTRIES (stream from SSE):
  Each entry is a row, entering from top with animation:
    Framer Motion: y: -8 → 0, opacity: 0 → 1, duration: 0.2s

  ENTRY TYPES (styled differently):

  AGENT_THINKING:
    Icon: brain icon (plasma color)
    Text: "Researcher-1 is analyzing the codebase..."
    Style: text-secondary, 12px

  TOOL_CALLED:
    Icon: wrench icon (synapse color)
    Text: "→ web_search: 'Next.js 15 streaming best practices'"
    Style: font-mono, 11px, synapse color
    Expandable: click to see full tool parameters

  TOOL_RESULT:
    Icon: check icon (signal color)
    Text: "← 6 sources found in 340ms"
    Style: font-mono, 11px, text-ghost
    Expandable: click to see results preview

  MESSAGE_SENT (agent → agent):
    Icon: arrow icon (ember color)
    Text: "Orchestrator → Executor: 'Implement the auth middleware'"
    Style: 11px, text-secondary
```

```
    Small indentation to show hierarchy

  AGENT_COMPLETED:
    Icon: check-circle (signal)
    Text: "✓ Researcher-1 completed in 12.4s (4,230 tokens)"
    Style: signal color, 12px, brief highlight flash

  HUMAN_CHECKPOINT:
    Special styling: full-width amber banner
    Icon: pause circle, ember color
    Text: "Agent is requesting human approval before proceeding"
    CTA: "Review & Approve" button + "Reject" button
    (These emit events back to Inngest waitForEvent)

SCROLL BEHAVIOR:
  Auto-scroll to bottom as new events arrive
  Pause on manual scroll (show "↓ Jump to live" floating button)
  Button: returns scroll to bottom + resumes auto-scroll

═══════════════════════════════════════════════════════════════
STEP 4 — AGENT STATUS CARDS (KIMI PARALLEL THREADS VIEW)
═══════════════════════════════════════════════════════════════

File: /components/workspace/AgentStatusCards.tsx

POSITION: Below the thought stream, or in a collapsible section

SHOWS: All spawned agents simultaneously (like Kimi's parallel thread
view)
LAYOUT: Stacked cards, compact (56px each)

EACH AGENT CARD:
  Left:
    Agent geometric avatar (unique shape per role):
      ORCHESTRATOR: hexagon (plasma)
      RESEARCHER:   circle (synapse)
      EXECUTOR:     square (ember)
      CRITIC:       triangle (signal)
      SYNTHESIZER:  diamond (purple)
    Small pulsing ring around avatar when RUNNING

  Center:
    Agent name (bold, 12px) + role badge
    Current task (truncated, 11px, text-secondary, italic)
    Progress bar if known (thin, 2px height, plasma fill)

  Right:
    Status (DataBadge)
    Token count (font-mono, 10px)

ACTIVE AGENT HIGHLIGHT:
```

```
  Currently running agent card: left border 3px plasma + surface-2 bg
  Completed: dimmed (opacity 0.6) but still visible
  Failed: ember left border, error icon

PHASE 3 COMPLETE CRITERIA:
✅ Plan DAG renders with correct dependency lines
✅ Thought stream receives SSE events and renders them correctly
✅ Entries animate in from top, auto-scroll works
✅ Human checkpoint banner appears and buttons emit correct events
✅ All 5 agent avatar shapes render distinctly
✅ Clicking a plan step expands to show live streaming output
```

╔═══╗
║ PHASE 4 — KIMI-STYLE RIGHT PANEL: SOURCES, MEMORY, ARTIFACTS ║
╚═══╝

```
AGENT ROLE: UI Engineer
OBJECTIVE: The right panel shows everything the swarm PRODUCES and
CONSUMES.
           Sources being read (Kimi-style), memory being built,
           artifacts produced, and final output.
AUTONOMY: Full auto-execute.

──────────────────────────────────────────────────────────────────

STEP 1 — RIGHT PANEL TAB SYSTEM

──────────────────────────────────────────────────────────────────

File: /components/workspace/RightPanel.tsx

4 tabs at top of right panel:
  📖 Sources  |  🧠 Memory  |  📁 Files  |  ✅ Output

Compact tab bar: 32px height, font-mono 11px

──────────────────────────────────────────────────────────────────

STEP 2 — SOURCES TAB (KIMI RESEARCH FEED)

──────────────────────────────────────────────────────────────────

File: /components/workspace/SourcesFeed.tsx
```

```
CONCEPT: As the Researcher agent calls web_search and read_url tools,
         sources appear here in real-time — just like Kimi shows
         "Reading 47 sources" as it works.

HEADER: "Sources" + live count badge (e.g., "12 sources")

SOURCE ENTRY (added in real-time as TOOL_RESULT events arrive):
  Animation: slides in from right (x: 20 → 0, opacity: 0 → 1)

  Layout per source:
    Favicon (16px, fetched from Google favicon API)
    Site name (bold, 12px, 1 line)
    Article title (11px, text-secondary, 2 lines max)
    Snippet excerpt (10px, text-ghost, 2 lines, font-body italic)
    Tags: relevance score chip + domain chip

  READING STATE:
    When agent is actively reading a URL:
    Card has animated plasma left border + pulsing background
    Text: "Reading..." in synapse color

  COMPLETED:
    Normal styling, checkmark badge top-right

SEARCH QUERY CHIPS (above source list):
  Each distinct search query the agent ran shown as a pill:
  Font-mono, 11px, plasma border, text-secondary
  Clicking a query pill filters the source list to that query's results

═══════════════════════════════════════════════════════════════════

STEP 3 — MEMORY TAB

═══════════════════════════════════════════════════════════════════

File: /components/workspace/MemoryPanel.tsx

Shows all memory_store tool calls from agents:
Key insights the swarm is building as it works

MEMORY ENTRY:
  Icon: bookmark icon (synapse)
  Key: font-mono, plasma color, 11px
  Value: text-primary, 12px, 3 lines max (expandable)
  Agent that stored it: text-ghost, 10px
  Tags: small chips (surface-3 background)
  Timestamp: relative (font-mono, 10px)

MEMORY GRAPH (compact visualization):
  SVG force graph showing memory entries as nodes
  Edges between entries that share tags
  Click a node: highlights the entry in the list below
  Size of node: proportional to how many times it was retrieved
```

```
═══════════════════════════════════════════════════════════════
STEP 4 — FILES TAB
═══════════════════════════════════════════════════════════════

File: /components/workspace/ArtifactsPanel.tsx

Tree view of files produced by the swarm session (from file_write tool):
  Style: VS Code file explorer feel (without the icons being too
literal)
  Each file: filename + size + agent that created it + timestamp
  Clicking a file: opens it in the center Monaco Editor tab
  Download button: per file (signed S3 URL)

FILE TYPE ICONS (colored geometric shapes, not emoji):
  .ts / .tsx:  plasma hexagon
  .py:         synapse circle
  .md:         signal square
  .json:       ember triangle
  .csv:        purple diamond
  other:       text-ghost dot

═══════════════════════════════════════════════════════════════
STEP 5 — OUTPUT TAB (FINAL SYNTHESIS)
═══════════════════════════════════════════════════════════════

File: /components/workspace/FinalOutput.tsx

WHILE SWARM IS RUNNING: Shows "Synthesis in progress..." with animated
  plasma loading indicator (orbital ring animation)

WHEN SWARM COMPLETES:
  Slides in with scaleIn animation

  Header: "Mission Complete" — signal color, display font
  Confidence score: "92% confidence" with arc gauge visual
  Time + Cost summary: "Completed in 4m 23s | Cost: $0.0089"

  CONTENT:
  Rendered markdown (react-markdown + rehype-highlight)
  Full synthesis output from the Synthesizer agent

  Section headers auto-detected and shown as mini table of contents
  (sidebar TOC within this panel, links to sections)

  COPY / EXPORT BAR (sticky bottom of panel):
    Copy to clipboard | Download .md | Open in Chat | Create New Session

PHASE 4 COMPLETE CRITERIA:
✅ Sources tab receives real-time entries as agents search
✅ Search query chips filter the source list
```

```
✅ Memory panel shows stored insights with tags
✅ Files panel shows tree with click-to-open in Monaco
✅ Output tab shows synthesis with markdown rendering
✅ All 4 right panel tabs switch correctly
```

╔═══╗ ‖ PHASE 5 — SWARM LAUNCHER: TASK SUBMISSION UI ‖ ╚═══╝

```
AGENT ROLE: Product UI Engineer
OBJECTIVE: The task submission experience must feel powerful.
           Not a text box. A mission briefing terminal.
           Like launching a rocket — deliberate, configured, committed.
AUTONOMY: Full auto-execute.

──────────────────────────────────────────────────────────────────
STEP 1 — SWARM LAUNCHER PAGE
──────────────────────────────────────────────────────────────────

File: /app/dashboard/swarm/new/page.tsx

FULL-PAGE LAYOUT:
  Background: --void with the radial plasma gradient from Phase 1 tokens
  Center column: max-width 720px, vertically centered

HEADER:
  Small icon: swarm hexagon SVG (animated — 6 dots orbiting a center)
  Title: "New Swarm Session" — display font, 40px
  Subtitle: "Define your objective. The swarm decomposes and executes."
— text-secondary

──────────────────────────────────────────────────────────────────
STEP 2 — OBJECTIVE INPUT
──────────────────────────────────────────────────────────────────

TEXTAREA — the primary input:
  Label: "Mission Objective" (font-mono, 11px, synapse color, uppercase)
  Height: 140px (auto-expands)
  PlasmaInput styling: full plasma glow on focus
  Placeholder: "e.g., Research the top 5 LLM routing strategies,
```

```
                implement a benchmarking script in Python,
                and produce a comparison report."
  Character counter: bottom-right, font-mono, 10px, text-ghost
                     turns ember at 80% of 2000 char limit

──────────────────────────────────────────────────────────────────

STEP 3 — SWARM CONFIGURATION PANEL
──────────────────────────────────────────────────────────────────

GlassCard below the objective textarea: "Swarm Configuration"

ROW 1 — Orchestrator Model:
  Label: "Orchestrator" — text-secondary, 12px
  Selector: pill group — Claude Sonnet | GPT-4o | Gemini Pro
  Active pill: plasma fill, white text
  Inactive: ghost style

ROW 2 — Worker Models (multi-select):
  Label: "Worker Pool"
  Checkboxes styled as model pills (same geometric icons as model
switcher)
  GPT-4o ✓ | Claude Haiku ✓ | Gemini Flash ✓ | Llama 3.3 ✓ | Kimi K2 □

ROW 3 — Max Agents & Max Steps:
  Two compact number inputs side by side
  "Max Parallel Agents": 1 → 10 (default 4)
  "Max Steps per Agent": 5 → 20 (default 10)

ROW 4 — Tools Allowed:
  Toggle chips: Web Search | Code Execute | Read URL | Memory | File
Write
  All ON by default, toggleable off

ROW 5 — Human Checkpoints:
  Toggle switch (on/off)
  ON: agent will pause at critical decisions for human review
  Tooltip: "Adds 10-30 min to session but ensures accuracy on sensitive
tasks"

──────────────────────────────────────────────────────────────────

STEP 4 — CREDIT ESTIMATE (before launch)
──────────────────────────────────────────────────────────────────

GlassCard: "Estimated Cost"
  Calculated from: objective complexity + agent count + models selected
  (simple heuristic: word count × agents × model cost tier)

  Shows: "~$0.0040 – $0.0120" range
  "~45 seconds – 3 minutes"
  Your balance: $X.XX remaining
  Warning if balance < estimated max cost: ember-colored banner
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
STEP 5 — LAUNCH BUTTON + ANIMATION
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

LAUNCH BUTTON:
  Full-width, height 56px
  Text: "Launch Swarm →" — display font, 18px
  Background: animated gradient (plasma → synapse → plasma, 3s loop)
  Border-radius: --radius-lg
  Hover: scale(1.01) + glow intensifies
  Active: scale(0.99)

ON CLICK:
  1. Button shows loading spinner + "Initializing swarm..."
  2. POST to /api/swarm/start
  3. Success: NeuralButton transforms → "Swarm Active ✓" (signal green,
800ms)
  4. Page transitions to /dashboard/workspace/[sessionId]
  5. Workspace opens with session already in PLANNING state

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
STEP 6 — QUICK LAUNCH TEMPLATES
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

Above the objective textarea: "Quick Start"
Row of 4 template cards (GlassCard, compact, 2 lines of text):

  🔬 Deep Research   — "Research a topic and produce a structured
report"
  💻 Build & Test    — "Write, test, and document a code solution"
  🔍 Data Analysis   — "Analyze data and produce insights with
visualizations"
  📋 Content Create  — "Research and write long-form content with
citations"

Clicking a template: pre-fills objective textarea with a template prompt
  and sets appropriate configuration (models, tools, agent count)

PHASE 5 COMPLETE CRITERIA:
✅ Objective textarea expands, character counter works
✅ Swarm configuration changes persist to POST body
✅ Credit estimate updates when configuration changes
✅ Launch button triggers API call and redirects to workspace
✅ Templates pre-fill the form correctly
✅ Form is keyboard-navigable (tab order correct)
```

╟ PHASE 6 — SWARM HISTORY DASHBOARD & SESSION MANAGEMENT ╢

```
AGENT ROLE: UI Engineer
OBJECTIVE: Past swarm sessions are valuable. Build the history view
           so users can revisit, clone, and analyze past runs.
AUTONOMY: Full auto-execute.

═══════════════════════════════════════════════════════════════
STEP 1 — SWARM HISTORY PAGE
═══════════════════════════════════════════════════════════════

File: /app/dashboard/swarm/page.tsx

LAYOUT: Standard dashboard page (sidebar + content)

HEADER ROW:
  Left: "Swarm Sessions" — display font, 28px
  Right: NeuralButton primary "New Swarm →"

FILTER ROW:
  Status filter: All | Running | Completed | Failed (pill group)
  Sort: Newest | Oldest | Most Expensive | Longest Running
  Search: PlasmaInput "Search sessions..."

═══════════════════════════════════════════════════════════════
STEP 2 — SESSION CARDS
═══════════════════════════════════════════════════════════════

Grid: 2 columns desktop, 1 column mobile

EACH SESSION CARD (GlassCard):
  Header row:
    Status badge (DataBadge) + time elapsed or duration
    "..." menu: Open | Clone | Delete | Export

  Title: first 80 chars of objective (truncated)

  Mini agent timeline (horizontal strip, 32px height):
    Each agent shown as a small colored dot in sequence
    Connecting lines between them (completed = solid, pending = dashed)
    Hovering a dot: tooltip with agent name + duration
```

```
  Stats row (font-mono, 11px):
    Agents spawned: N
    Total tokens: X,XXX
    Cost: $0.00XX
    Duration: Xm Xs

  Sources used (if research session):
    "Read 12 sources" with favicon strip (first 5 favicons)

  Files produced:
    File count chip: "3 files" with file type icons

RUNNING SESSIONS:
  Card has animated plasma border (dashed, rotating)
  Live elapsed timer counting up
  "Open Live →" button (primary, full width)

──────────────────────────────────────────────────────────────
STEP 3 — ANALYTICS STRIP (TOP OF HISTORY PAGE)
──────────────────────────────────────────────────────────────

3 compact KPI cards before the session list:

"Sessions This Month": count + sparkline (7 days)
"Total AI Cost":       $ amount + trend vs last month
"Avg Session Time":    duration + fastest/slowest

PHASE 6 COMPLETE CRITERIA:
✅ History page loads with sessions from /api/swarm paginated
✅ Filter + sort works client-side (or via API query params)
✅ Session cards show correct status, cost, agent count
✅ Clone action pre-fills the new swarm form with same objective +
config
✅ Running sessions show live elapsed time
✅ Export action downloads session as JSON or markdown
```

╔═══╗ ‖ PHASE 7 — ADVANCED SWARM BACKEND: INTELLIGENCE LAYER ‖ ╚═══╝

```
AGENT ROLE: AI Systems Engineer
OBJECTIVE: Elevate the swarm from a basic multi-agent loop to a
           genuinely intelligent system: adaptive planning,
           critic feedback loops, self-healing, and cross-session
memory.
AUTONOMY: Full auto-execute on code. Pause on schema changes.

═══════════════════════════════════════════════════════════════
STEP 1 — CRITIC-IN-THE-LOOP PATTERN
═══════════════════════════════════════════════════════════════

Create: /lib/swarm/critic.ts

After each Executor agent completes, automatically spawn a Critic agent:

CRITIC PROMPT PATTERN:
  "You are a quality control critic. Review the output of
[EXECUTOR_NAME]
   for the following task: [TASK DESCRIPTION]

   Output to review:
   [EXECUTOR_OUTPUT]

   Evaluate:
   1. Correctness: Is the output factually/technically correct?
   2. Completeness: Does it fully address the task?
   3. Quality: Is it production-ready or rough?

   Return JSON: {
     approved: boolean,
     score: 0-100,
     issues: string[],
     suggestions: string[]
   }"

IF critic.approved = false AND critic.score < 70:
  → Spawn a Revision agent with the critic's feedback as context
  → Revision agent regenerates the output
  → Pass to critic again (max 2 revision cycles to prevent infinite
loops)

IF critic.approved = true OR score >= 70:
  → Accept output, continue to next step

═══════════════════════════════════════════════════════════════
STEP 2 — ADAPTIVE RE-PLANNING
═══════════════════════════════════════════════════════════════

Create: /lib/swarm/replanner.ts
```

```
If a step fails after 3 retries:
  → Call Orchestrator again with:
    - Original objective
    - Steps completed so far (their outputs)
    - Failed step details + error reason
    - Instruction: "Revise the remaining plan to achieve the objective
                    given what we know so far. The failed step was: [X].
                    Produce an alternative approach."
  → Orchestrator returns a revised partial plan
  → Resume execution with the revised steps
  → Emit: PLAN_REVISED event (visible in the thought stream)

═══════════════════════════════════════════════════════════════
STEP 3 — CROSS-SESSION MEMORY (pgvector)
═══════════════════════════════════════════════════════════════

Add to schema.prisma:

model SwarmMemoryVector {
  id          String   @id @default(cuid())
  userId      String
  sessionId   String
  content     String   @db.Text
  tags        String[]
  vector      Unsupported("vector(1536)")?
  createdAt   DateTime @default(now())
  user        User     @relation(fields: [userId], references: [id])

  @@index([userId])
}

Enable pgvector in Neon:
  CREATE EXTENSION IF NOT EXISTS vector;
  CREATE INDEX ON "SwarmMemoryVector" USING ivfflat (vector
vector_cosine_ops);

At session start:
  Retrieve top 5 semantically similar memories from past sessions
  Inject into Orchestrator's system prompt as context:
    "From previous sessions, you may find this relevant: [memories]"

At session end:
  Extract 3-5 key learnings from the synthesizer's output
  Store as vectors (embed via OpenAI text-embedding-3-small)
  Tag with: domain, task type, outcome quality

═══════════════════════════════════════════════════════════════
STEP 4 — SWARM PERFORMANCE ANALYTICS API
═══════════════════════════════════════════════════════════════

Create: /app/api/swarm/analytics/route.ts
```

```
Returns aggregated metrics:
  {
    totalSessions: number,
    successRate: number,           // % of sessions that COMPLETED
    avgDurationMs: number,
    avgCostUsd: number,
    avgAgentsPerSession: number,
    avgTokensPerSession: number,
    mostUsedModels: { model: string, count: number }[],
    criticsApprovalRate: number,   // % critic passes on first try
    replansTriggered: number,      // how often adaptive replanning
fires
    topFailureReasons: string[],   // from failed SwarmEvents
  }

═══════════════════════════════════════════════════════════════
STEP 5 — RATE LIMITS & COST GUARDRAILS
═══════════════════════════════════════════════════════════════

Implement in: /lib/swarm/guardrails.ts

HARD STOPS (auto-cancel session if exceeded):
  - Session cost exceeds user-set budget cap (stored in session config)
  - Session duration exceeds 30 minutes
  - Single agent exceeds 50,000 tokens
  - Total swarm exceeds 200,000 tokens
  - Recursive agent depth > 3 levels (call_agent spawning call_agent)

SOFT WARNINGS (emit warning event, continue):
  - Session cost > 80% of budget cap
  - Session duration > 20 minutes
  - Agent has iterated > 8 tool calls without completing

RATE LIMITS (Upstash Redis sliding window):
  - 5 concurrent swarm sessions per user (plan: Pro)
  - 2 concurrent swarm sessions per user (plan: Starter)
  - 10 swarm starts per hour per user
  - 1,000,000 tokens per user per day

PHASE 7 COMPLETE CRITERIA:
✅ Critic agent spawns after each Executor, approves or triggers
revision
✅ Failed steps trigger re-planning (test by deliberately failing a
step)
✅ pgvector extension active, memories stored and retrieved
✅ Cross-session context injected into Orchestrator prompt
✅ Cost guardrail cancels a session when budget exceeded
✅ Analytics endpoint returns correct aggregated data
```

╔═══ PHASE 8 — FINAL POLISH: MAKING IT FEEL ALIVE ═══╗

```
AGENT ROLE: UI Polish Engineer + Experience Designer
OBJECTIVE: The difference between "impressive demo" and "product people
           pay for" is feel. This phase makes it feel alive, reactive,
           and trustworthy.
AUTONOMY: Full auto-execute.

═══════════════════════════════════════════════════════════════
STEP 1 — WORKSPACE SOUND DESIGN (optional but powerful)
═══════════════════════════════════════════════════════════════

Use: Tone.js (already in stack from previous master query)

Subtle UI sounds (OFF by default, toggle in settings):
  Agent spawned:   soft synth blip (220hz, 80ms, sine)
  Tool called:     click sound (short noise burst, 30ms)
  Agent complete:  soft ascending 2-note chime
  Swarm complete:  satisfying 3-note resolution chord
  Error:           low descending tone

Volume: 15% default when enabled
Respect prefers-reduced-motion: also silence sounds

═══════════════════════════════════════════════════════════════
STEP 2 — LIVE ACTIVITY INDICATORS
═══════════════════════════════════════════════════════════════

BROWSER TAB:
  While swarm is running: tab title animates:
  "⚡ AIVerse — Running (3/7 steps)" — updates every 10s
  On complete: "✓ AIVerse — Mission Complete"
  On failure:  "✗ AIVerse — Session Failed"

FAVICON:
  Use canvas to draw dynamic favicon:
  IDLE: normal logo
  RUNNING: logo with small animated plasma dot overlay
  COMPLETE: logo with green checkmark overlay
```

```
═══════════════════════════════════════════════════════════════
STEP 3 — KEYBOARD SHORTCUT SYSTEM
═══════════════════════════════════════════════════════════════

Create: /lib/keyboard-shortcuts.ts

Implement with: react-hotkeys-hook

GLOBAL SHORTCUTS:
  CMD+K:          Open global search / command palette
  CMD+N:          New swarm session
  CMD+Shift+T:    Open terminal tab in workspace
  CMD+Shift+E:    Open editor tab in workspace
  ESC:            Close any open modal/panel
  Space (on session list): open selected session
  P (in workspace): pause/resume swarm

COMMAND PALETTE (CMD+K):
  GlassCard modal, centered, 600px wide
  Search input at top (PlasmaInput, autofocus)
  Results: recent sessions + actions + navigation
  Items: icon + label + keyboard shortcut badge
  Arrow keys to navigate, Enter to execute

═══════════════════════════════════════════════════════════════
STEP 4 — FINAL PRODUCTION CHECKLIST
═══════════════════════════════════════════════════════════════

Run all and fix until passing:

BUILD CHECKS:
  □ npm run build — zero errors, zero warnings
  □ npm run lint — ESLint strict, zero violations
  □ npx tsc --noEmit — TypeScript strict, zero errors
  □ npm audit — zero HIGH/CRITICAL vulnerabilities

PERFORMANCE:
  □ Lighthouse Performance > 85 (desktop), > 70 (mobile)
  □ Workspace panel resize is smooth (no layout jank)
  □ xterm.js renders 1000 lines without lag
  □ Monaco editor loads < 2s (code-split correctly)
  □ SSE connection stays alive through 10+ minute session

SECURITY:
  □ All /api/swarm/* routes require auth
  □ Users cannot access other users' sessions
  □ Cost guardrails cannot be bypassed via API
  □ No secrets in any client-side bundle (check with: grep -r "sk-"
.next/)

EXPERIENCE:
```

```
  □ New swarm → workspace transition is seamless (no blank screen)
  □ Thought stream auto-scrolls, pauses on hover, resumes on ↓ button
  □ Human checkpoint banner blocks until decision is made
  □ Session cards on history page reflect correct status
  □ Export downloads valid markdown/JSON
  □ Clone pre-fills new session form correctly

PHASE 8 = PRODUCT SHIPPED ✅
```

EXECUTION SUMMARY

Phase	Area	Core Deliverable
1	Swarm Backend	Prisma models, Orchestrator, Inngest jobs, SSE, Tools
2	IDE Shell UI	4-panel resizable workspace, Monaco, xterm.js
3	Agent Rail (Manus)	Plan DAG, thought stream, agent status cards
4	Right Panel (Kimi)	Sources feed, memory, files tree, final output
5	Swarm Launcher	Mission briefing form, config, credit estimate
6	Session History	History grid, clone, export, analytics strip
7	Intelligence Layer	Critic loop, adaptive re-planning, pgvector memory
8	Polish & Ship	Sound, live favicon, keyboard shortcuts, final QA

NORTH STAR: What this product looks like when done

→ User types: *"Research the top 5 AI coding assistants, benchmark them on 10 coding tasks, write a Python benchmarking script, and produce a comparison report with visualizations."*

→ Swarm launches. Left rail shows plan decomposing into 6 steps.
Researcher-1 and Researcher-2 spawn simultaneously (parallel).
Right panel fills with sources being read in real-time.
Monaco editor opens as Executor-1 writes Python code live.
Terminal streams test output as code runs.
Critic reviews the code. Flags one issue. Revision agent fixes it.
Synthesizer assembles everything into a 2,000-word report.

Output tab shows final markdown. Files tab shows benchmark.py + report.md.
Total: 4 minutes. Cost: $0.011.

That is what we are building.

AIVerse 2.0 — Swarm Master Query | For Mahadeva | Solo Founder Build